



```
In [1]: # =====
# CELL 1
# IMPORT LIBRARY
# =====

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import joblib

from tensorflow.keras.models import load_model

print("=" * 75)
print("IMPORT LIBRARY")
print("=" * 75)

print("✓ NumPy")
print("✓ Pandas")
print("✓ Joblib")
print("✓ TensorFlow / Keras")
print()

print("Status")
print("✓ Seluruh library berhasil diimpor.")
print("=" * 75)
```

```
=====
IMPORT LIBRARY
=====
✓ NumPy
✓ Pandas
✓ Joblib
✓ TensorFlow / Keras

Status
✓ Seluruh library berhasil diimpor.
=====
```

```
In [2]: from google.colab import drive
```

```
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

```
In [3]: # =====
# CELL 2
# LOAD MODEL
# =====

from pathlib import Path
```

```

print("=" * 75)
print("LOAD MODEL")
print("=" * 75)

# =====
# Lokasi Model
# =====

MODEL_PATH = Path("/content/drive/MyDrive/Skripsi/models/gab_window8.keras")
# Ganti sesuai lokasi model Anda

# =====
# Cek File
# =====

if not MODEL_PATH.exists():
    raise FileNotFoundError(
        f"Model tidak ditemukan:\n{MODEL_PATH}"
    )

print(f"Model ditemukan : {MODEL_PATH.name}")

# =====
# Load Model
# =====

model = load_model(MODEL_PATH)

print("\n✓ Model berhasil dimuat.")

# =====
# Informasi Model
# =====

print("\nInput Shape")
print(model.input_shape)

print("\nOutput Shape")
print(model.output_shape)

print("\nModel Summary")
model.summary()

print("=" * 75)

```

```
=====
LOAD MODEL
=====
```

Model ditemukan : gab_window8.keras

✓ Model berhasil dimuat.

Input Shape
(None, 8, 11)

Output Shape
(None, 2)

Model Summary

Model: "sequential_8"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	19,456
dense_hidden (Dense)	(None, 32)	2,080
output (Dense)	(None, 2)	66

Total params: 64,808 (253.16 KB)

Trainable params: 21,602 (84.38 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 43,206 (168.78 KB)

```
=====
In [4]: # =====
# CELL 3
# LOAD FEATURE & TARGET SCALER
# =====

from pathlib import Path
import joblib

print("=" * 80)
print("LOAD FEATURE & TARGET SCALER")
print("=" * 80)

# =====
# Folder Scaler
# =====

SCALER_DIR = Path(
    "/content/drive/MyDrive/Skripsi/normalized_dataset"
)

FEATURE_SCALER_PATH = SCALER_DIR / "feature_scaler_gab.pkl"
TARGET_SCALER_PATH = SCALER_DIR / "target_scaler_gab.pkl"
```

```

# =====
# Cek File
# =====

assert FEATURE_SCALER_PATH.exists(), \
    f"Feature scaler tidak ditemukan:\n{FEATURE_SCALER_PATH}"

assert TARGET_SCALER_PATH.exists(), \
    f"Target scaler tidak ditemukan:\n{TARGET_SCALER_PATH}"

# =====
# Load
# =====

feature_scaler = joblib.load(FEATURE_SCALER_PATH)
target_scaler = joblib.load(TARGET_SCALER_PATH)

print(f"✓ Feature Scaler : {FEATURE_SCALER_PATH.name}")
print(f"✓ Target Scaler : {TARGET_SCALER_PATH.name}")

# =====
# Informasi
# =====

print("\nJumlah Feature")

if hasattr(feature_scaler, "n_features_in_"):
    print(feature_scaler.n_features_in_)

print("\nJumlah Target")

if hasattr(target_scaler, "n_features_in_"):
    print(target_scaler.n_features_in_)

print("=" * 80)

```

```

=====
=
LOAD FEATURE & TARGET SCALER
=====
=
✓ Feature Scaler : feature_scaler_gab.pkl
✓ Target Scaler : target_scaler_gab.pkl

Jumlah Feature
9

Jumlah Target
2
=====
=

```

In [5]: # =====

```

# CELL 4
# INFERENCE CONFIGURATION
# =====

print("=" * 80)
print("INFERENCE CONFIGURATION")
print("=" * 80)

# =====
# URUTAN FEATURE YANG MASUK KE MODEL
# (HARUS SAMA PERSIS SAAT TRAINING)
# =====

FEATURE_COLUMNS = [

    "LAT",
    "LON",

    "WMO_WIND",
    "WMO_PRES",

    "delta_lat",
    "delta_lon",

    "speed_kmh",
    "bearing_rate",
    "acceleration",

    "sin_month",
    "cos_month"

]

# =====
# FEATURE YANG DISTANDARDISASI
# =====

FEATURES_TO_SCALE = [

    "WMO_WIND",
    "WMO_PRES",

    "delta_lat",
    "delta_lon",

    "speed_kmh",
    "bearing_rate",
    "acceleration",

    "sin_month",
    "cos_month"

]

```

```

# =====
# TARGET MODEL
# =====

TARGET_COLUMNS = [

    "LAT",
    "LON"

]

# =====
# WINDOW MODEL
# =====

WINDOW_SIZE = 8

# =====
# VALIDASI
# =====

print(f"Window Size           : {WINDOW_SIZE}")
print(f"Total Feature Model     : {len(FEATURE_COLUMNS)}")
print(f"Total Feature Scaler      : {len(FEATURES_TO_SCALE)}")
print(f"Total Target               : {len(TARGET_COLUMNS)}")

print("\nUrutan Feature Model")

for i, col in enumerate(FEATURE_COLUMNS, start=1):
    print(f"{i:02d}. {col}")

print("\nFeature yang di-StandardScaler")

for i, col in enumerate(FEATURES_TO_SCALE, start=1):
    print(f"{i:02d}. {col}")

print("\nTarget Prediksi")

for col in TARGET_COLUMNS:
    print(f"• {col}")

print("=" * 80)
print("✓ Konfigurasi inference siap digunakan.")
print("=" * 80)

```

```
=====
=
INFERENCE CONFIGURATION
=====
=
Window Size          : 8
Total Feature Model   : 11
Total Feature Scaler  : 9
Total Target          : 2

Urutan Feature Model
01. LAT
02. LON
03. WMO_WIND
04. WMO_PRES
05. delta_lat
06. delta_lon
07. speed_kmh
08. bearing_rate
09. acceleration
10. sin_month
11. cos_month

Feature yang di-StandardScaler
01. WMO_WIND
02. WMO_PRES
03. delta_lat
04. delta_lon
05. speed_kmh
06. bearing_rate
07. acceleration
08. sin_month
09. cos_month

Target Prediksi
• LAT
• LON
=====
=
✓ Konfigurasi inference siap digunakan.
=====
=
```

```
In [6]: # =====
# CELL 5
# INPUT MANUAL (8 TIMESTEP)
# =====

print("=" * 80)
print("INPUT DATA MANUAL")
print("=" * 80)

# =====
# MASUKKAN 8 DATA TERAKHIR
```

```

# Urutkan dari data PALING LAMA -> PALING BARU
# Interval waktu harus sama seperti data training
# (contoh di bawah menggunakan interval 3 jam)
# =====

input_data = pd.DataFrame({

    "ISO_TIME": [

        "2024-01-01 00:00",
        "2024-01-01 03:00",
        "2024-01-01 06:00",
        "2024-01-01 09:00",
        "2024-01-01 12:00",
        "2024-01-01 15:00",
        "2024-01-01 18:00",
        "2024-01-01 21:00"

    ],

    "LAT": [

        -8.20,
        -8.10,
        -8.00,
        -7.90,
        -7.80,
        -7.70,
        -7.60,
        -7.50

    ],

    "LON": [

        112.10,
        112.30,
        112.50,
        112.70,
        112.90,
        113.10,
        113.30,
        113.50

    ],

    "WMO_WIND": [

        55,
        60,
        60,
        65,
        65,

    ]
})

```



```

        70,
        70,
        75

    ],

    "WMO_PRES": [

        996,
        994,
        992,
        990,
        988,
        986,
        984,
        982

    ]

})

# =====
# Konversi waktu
# =====

input_data["ISO_TIME"] = pd.to_datetime(
    input_data["ISO_TIME"]
)

# =====
# Validasi
# =====

print(input_data)

print("\nJumlah Data :", len(input_data))

assert len(input_data) == WINDOW_SIZE, \
    f"Jumlah data harus {WINDOW_SIZE}"

print("\nRentang Waktu")
print(
    input_data["ISO_TIME"].min(),
    "s/d",
    input_data["ISO_TIME"].max()
)

print("\n✓ Input valid.")
print("=" * 80)

```

```
=====
=
INPUT DATA MANUAL
=====
=
```

	ISO_TIME	LAT	LON	WMO_WIND	WMO_PRES
0	2024-01-01 00:00:00	-8.2	112.1	55	996
1	2024-01-01 03:00:00	-8.1	112.3	60	994
2	2024-01-01 06:00:00	-8.0	112.5	60	992
3	2024-01-01 09:00:00	-7.9	112.7	65	990
4	2024-01-01 12:00:00	-7.8	112.9	65	988
5	2024-01-01 15:00:00	-7.7	113.1	70	986
6	2024-01-01 18:00:00	-7.6	113.3	70	984
7	2024-01-01 21:00:00	-7.5	113.5	75	982

Jumlah Data : 8

Rentang Waktu
2024-01-01 00:00:00 s/d 2024-01-01 21:00:00

✓ Input valid.

```
=====
=
```

```
In [7]: # =====
# CELL 6
# UTILITY FUNCTION
# =====

import numpy as np

print("=" * 80)
print("UTILITY FUNCTION")
print("=" * 80)

# =====
# HAVERSINE DISTANCE (KM)
# =====

def haversine(lat1, lon1, lat2, lon2):

    R = 6371.0

    lat1 = np.radians(lat1)
    lon1 = np.radians(lon1)
    lat2 = np.radians(lat2)
    lon2 = np.radians(lon2)

    dlat = lat2 - lat1
    dlon = lon2 - lon1

    a = (
        np.sin(dlat / 2) ** 2
        + np.cos(lat1)
```

```

        * np.cos(lat2)
        * np.sin(dlon / 2) ** 2
    )

    c = 2 * np.arcsin(np.sqrt(a))

    return R * c

# =====
# BEARING
# =====

def bearing(lat1, lon1, lat2, lon2):

    lat1 = np.radians(lat1)
    lon1 = np.radians(lon1)

    lat2 = np.radians(lat2)
    lon2 = np.radians(lon2)

    dlon = lon2 - lon1

    x = np.sin(dlon) * np.cos(lat2)

    y = (
        np.cos(lat1) * np.sin(lat2)
        - np.sin(lat1)
        * np.cos(lat2)
        * np.cos(dlon)
    )

    angle = np.degrees(np.arctan2(x, y))

    return (angle + 360) % 360

# =====
# ANGLE DIFFERENCE
# =====

def angle_diff(series):

    diff = series.diff()

    diff = (diff + 180) % 360 - 180

    return diff.abs()

print("✓ haversine()")
print("✓ bearing()")
print("✓ angle_diff()")

print("=" * 80)

```

```
=====
=
UTILITY FUNCTION
=====
=
✓ haversine()
✓ bearing()
✓ angle_diff()
=====
=
```

```
In [8]: # =====
# CELL 7
# FEATURE ENGINEERING (INFERENCE)
# =====

print("=" * 80)
print("FEATURE ENGINEERING")
print("=" * 80)

# =====
# Salin Data
# =====

feature_data = input_data.copy()

# =====
# Previous Observation
# =====

prev_lat = feature_data["LAT"].shift(1)
prev_lon = feature_data["LON"].shift(1)
prev_time = feature_data["ISO_TIME"].shift(1)

# =====
# Delta Time (Hour)
# =====

delta_hour = (
    feature_data["ISO_TIME"] - prev_time
).dt.total_seconds() / 3600

# =====
# Delta Coordinate
# =====

feature_data["delta_lat"] = (
    feature_data["LAT"] - prev_lat
)

feature_data["delta_lon"] = (
    feature_data["LON"] - prev_lon
)
```

```

# =====
# Distance (km)
# =====

distance = haversine(
    prev_lat,
    prev_lon,
    feature_data["LAT"],
    feature_data["LON"]
)

# =====
# Speed (km/h)
# =====

feature_data["speed_kmh"] = np.where(
    delta_hour > 0,
    distance / delta_hour,
    np.nan
)

# =====
# Bearing
# =====

feature_data["bearing"] = bearing(
    prev_lat,
    prev_lon,
    feature_data["LAT"],
    feature_data["LON"]
)

# =====
# Bearing Rate
# =====

feature_data["bearing_rate"] = angle_diff(
    feature_data["bearing"]
)

# =====
# Acceleration
# =====

prev_speed = feature_data["speed_kmh"].shift(1)

feature_data["acceleration"] = np.where(
    delta_hour > 0,
    (feature_data["speed_kmh"] - prev_speed) / delta_hour,
    np.nan
)

# =====

```

```

# Seasonal Feature
# =====

month = feature_data["ISO_TIME"].dt.month

feature_data["sin_month"] = np.sin(
    2 * np.pi * month / 12
)

feature_data["cos_month"] = np.cos(
    2 * np.pi * month / 12
)

# =====
# Isi Nilai Awal
# (Tidak ada previous observation)
# =====

feature_data[
    [
        "delta_lat",
        "delta_lon",
        "speed_kmh",
        "bearing_rate",
        "acceleration"
    ]
] = feature_data[
    [
        "delta_lat",
        "delta_lon",
        "speed_kmh",
        "bearing_rate",
        "acceleration"
    ]
].fillna(0)

# =====
# Hapus Kolom Sementara
# =====

feature_data.drop(
    columns=["bearing"],
    inplace=True
)

# =====
# Tampilkan Hasil
# =====

print(feature_data)

print("\nJumlah Data :", len(feature_data))

```

```

print("\nFeature yang dihasilkan:")

for col in FEATURE_COLUMNS:
    print(f"✓ {col}")

print("=" * 80)
print("✓ Feature Engineering selesai.")
print("=" * 80)

```

FEATURE ENGINEERING

	ISO_TIME	LAT	LON	WMO_WIND	WMO_PRES	delta_lat	delta_lon	\
0	2024-01-01 00:00:00	-8.2	112.1	55	996	0.0	0.0	
1	2024-01-01 03:00:00	-8.1	112.3	60	994	0.1	0.2	
2	2024-01-01 06:00:00	-8.0	112.5	60	992	0.1	0.2	
3	2024-01-01 09:00:00	-7.9	112.7	65	990	0.1	0.2	
4	2024-01-01 12:00:00	-7.8	112.9	65	988	0.1	0.2	
5	2024-01-01 15:00:00	-7.7	113.1	70	986	0.1	0.2	
6	2024-01-01 18:00:00	-7.6	113.3	70	984	0.1	0.2	
7	2024-01-01 21:00:00	-7.5	113.5	75	982	0.1	0.2	

	speed_kmh	bearing_rate	acceleration	sin_month	cos_month
0	0.000000	0.000000	0.000000	0.5	0.866025
1	8.221083	0.000000	0.000000	0.5	0.866025
2	8.222710	0.005554	0.000542	0.5	0.866025
3	8.224318	0.005482	0.000536	0.5	0.866025
4	8.225905	0.005409	0.000529	0.5	0.866025
5	8.227473	0.005337	0.000523	0.5	0.866025
6	8.229020	0.005265	0.000516	0.5	0.866025
7	8.230548	0.005192	0.000509	0.5	0.866025

Jumlah Data : 8

Feature yang dihasilkan:

- ✓ LAT
- ✓ LON
- ✓ WMO_WIND
- ✓ WMO_PRES
- ✓ delta_lat
- ✓ delta_lon
- ✓ speed_kmh
- ✓ bearing_rate
- ✓ acceleration
- ✓ sin_month
- ✓ cos_month

✓ Feature Engineering selesai.

```
In [9]: # =====
# CELL 8
# FEATURE SCALING
# =====

print("=" * 80)
print("FEATURE SCALING")
print("=" * 80)

# =====
# Salin Data
# =====

scaled_data = feature_data.copy()

# =====
# Standardisasi 9 Feature
# =====

scaled_data[FEATURES_TO_SCALE] = feature_scaler.transform(
    scaled_data[FEATURES_TO_SCALE]
)

print(scaled_data)

print("\nDimensi Data :", scaled_data.shape)

print("\nFeature yang di-standardisasi")

for col in FEATURES_TO_SCALE:
    print(f"✓ {col}")

print("\nLAT dan LON tetap nilai asli")

print("=" * 80)
print("✓ Feature Scaling selesai.")
print("=" * 80)
```



```
=====
=
FEATURE SCALING
=====
```

```
=
      ISO_TIME  LAT    LON  WMO_WIND  WMO_PRES  delta_lat  delta_lon  \
0 2024-01-01 00:00:00 -8.2  112.1  0.755176  0.339132  0.372593  0.311801
1 2024-01-01 03:00:00 -8.1  112.3  0.999559  0.219929  0.746808  0.823095
2 2024-01-01 06:00:00 -8.0  112.5  0.999559  0.100726  0.746808  0.823095
3 2024-01-01 09:00:00 -7.9  112.7  1.243942 -0.018477  0.746808  0.823095
4 2024-01-01 12:00:00 -7.8  112.9  1.243942 -0.137680  0.746808  0.823095
5 2024-01-01 15:00:00 -7.7  113.1  1.488325 -0.256883  0.746808  0.823095
6 2024-01-01 18:00:00 -7.6  113.3  1.488325 -0.376086  0.746808  0.823095
7 2024-01-01 21:00:00 -7.5  113.5  1.732708 -0.495289  0.746808  0.823095
```

```
      speed_kmh  bearing_rate  acceleration  sin_month  cos_month
0  -1.479623    -0.674620    -0.004210    0.237385    0.859743
1  -0.684506    -0.674620    -0.004210    0.237385    0.859743
2  -0.684349    -0.674407    -0.004208    0.237385    0.859743
3  -0.684193    -0.674409    -0.004208    0.237385    0.859743
4  -0.684040    -0.674412    -0.004208    0.237385    0.859743
5  -0.683888    -0.674415    -0.004208    0.237385    0.859743
6  -0.683739    -0.674418    -0.004208    0.237385    0.859743
7  -0.683591    -0.674421    -0.004208    0.237385    0.859743
```

Dimensi Data : (8, 12)

Feature yang di-standardisasi

- ✓ WMO_WIND
- ✓ WMO_PRES
- ✓ delta_lat
- ✓ delta_lon
- ✓ speed_kmh
- ✓ bearing_rate
- ✓ acceleration
- ✓ sin_month
- ✓ cos_month

LAT dan LON tetap nilai asli

```
=====
=
✓ Feature Scaling selesai.
=====
=
```

```
In [14]: # =====
# CELL 9
# BUILD MODEL INPUT
# =====

print("=" * 80)
print("BUILD MODEL INPUT")
print("=" * 80)
```

```

# =====
# URUTAN FEATURE HARUS SAMA DENGAN TRAINING
# =====

MODEL_FEATURES = [

    "LAT",
    "LON",

    "WMO_WIND",
    "WMO_PRES",

    "delta_lat",
    "delta_lon",

    "speed_kmh",
    "bearing_rate",
    "acceleration",

    "sin_month",
    "cos_month"

]

# =====
# AMBIL HANYA FEATURE MODEL
# =====

X = scaled_data[MODEL_FEATURES].copy()

print("Data Model")
print(X)

print("\nShape DataFrame :", X.shape)

# =====
# VALIDASI
# =====

assert list(X.columns) == MODEL_FEATURES, \
    "Urutan feature tidak sesuai!"

assert X.shape[0] == WINDOW_SIZE, \
    f"Jumlah timestep harus {WINDOW_SIZE}"

assert X.shape[1] == len(MODEL_FEATURES), \
    f"Jumlah feature harus {len(MODEL_FEATURES)}"

# =====
# UBAH MENJADI NUMPY
# =====

X = X.astype(np.float32).to_numpy()

```

```
print("\nShape Numpy :", X.shape)

# =====
# RESHAPE UNTUK LSTM
# =====

X = X.reshape(1, WINDOW_SIZE, len(MODEL_FEATURES))

print("Shape Input Model :", X.shape)

# =====
# VALIDASI INPUT MODEL
# =====

assert X.shape == (
    1,
    WINDOW_SIZE,
    len(MODEL_FEATURES)
), "Shape input model salah!"

print("\nInput Shape Model :", model.input_shape)

print("\n✓ Input siap diprediksi")

print("=" * 80)
```

```
=====
=
BUILD MODEL INPUT
=====
```

```
=
Data Model
  LAT    LON  WMO_WIND  WMO PRES  delta_lat  delta_lon  speed_kmh  \
0 -8.2   112.1  0.755176  0.339132  0.372593   0.311801  -1.479623
1 -8.1   112.3  0.999559  0.219929  0.746808   0.823095  -0.684506
2 -8.0   112.5  0.999559  0.100726  0.746808   0.823095  -0.684349
3 -7.9   112.7  1.243942 -0.018477  0.746808   0.823095  -0.684193
4 -7.8   112.9  1.243942 -0.137680  0.746808   0.823095  -0.684040
5 -7.7   113.1  1.488325 -0.256883  0.746808   0.823095  -0.683888
6 -7.6   113.3  1.488325 -0.376086  0.746808   0.823095  -0.683739
7 -7.5   113.5  1.732708 -0.495289  0.746808   0.823095  -0.683591
```

```
  bearing_rate  acceleration  sin_month  cos_month
0   -0.674620     -0.004210   0.237385   0.859743
1   -0.674620     -0.004210   0.237385   0.859743
2   -0.674407     -0.004208   0.237385   0.859743
3   -0.674409     -0.004208   0.237385   0.859743
4   -0.674412     -0.004208   0.237385   0.859743
5   -0.674415     -0.004208   0.237385   0.859743
6   -0.674418     -0.004208   0.237385   0.859743
7   -0.674421     -0.004208   0.237385   0.859743
```

Shape DataFrame : (8, 11)

Shape Numpy : (8, 11)

Shape Input Model : (1, 8, 11)

Input Shape Model : (None, 8, 11)

✓ Input siap diprediksi

```
=====
=
```

```
In [16]: # =====
# CELL 11
# VISUALISASI LINTASAN
# =====

import matplotlib.pyplot as plt

print("=" * 80)
print("VISUALISASI LINTASAN")
print("=" * 80)

# =====
# DATA HISTORI
# =====

history_lat = input_data["LAT"].values
history_lon = input_data["LON"].values
```

```

# =====
# HASIL PREDIKSI
# =====

pred_lat = float(y_pred_scaled[0, 0])
pred_lon = float(y_pred_scaled[0, 1])

# =====
# PLOT
# =====

plt.figure(figsize=(8,8))

# Histori
plt.plot(
    history_lon,
    history_lat,
    '-o',
    linewidth=2,
    markersize=6,
    label="8 Titik Historis"
)

# Titik terakhir
plt.scatter(
    history_lon[-1],
    history_lat[-1],
    s=120,
    marker='s',
    label="Posisi Terakhir"
)

# Prediksi
plt.scatter(
    pred_lon,
    pred_lat,
    s=180,
    marker='*',
    label="Prediksi LSTM"
)

# Garis menuju prediksi
plt.plot(
    [history_lon[-1], pred_lon],
    [history_lat[-1], pred_lat],
    '--',
    linewidth=2
)

# =====
# Nomor tiap titik
# =====

```

```

for i in range(len(history_lat)):
    plt.text(
        history_lon[i],
        history_lat[i],
        str(i+1),
        fontsize=9
    )

plt.text(
    pred_lon,
    pred_lat,
    "Pred",
    fontsize=10,
    weight="bold"
)

plt.xlabel("Longitude")
plt.ylabel("Latitude")

plt.title("Testing Prediksi Lintasan Siklon")

plt.grid(True)

plt.legend()

plt.axis("equal")

plt.show()

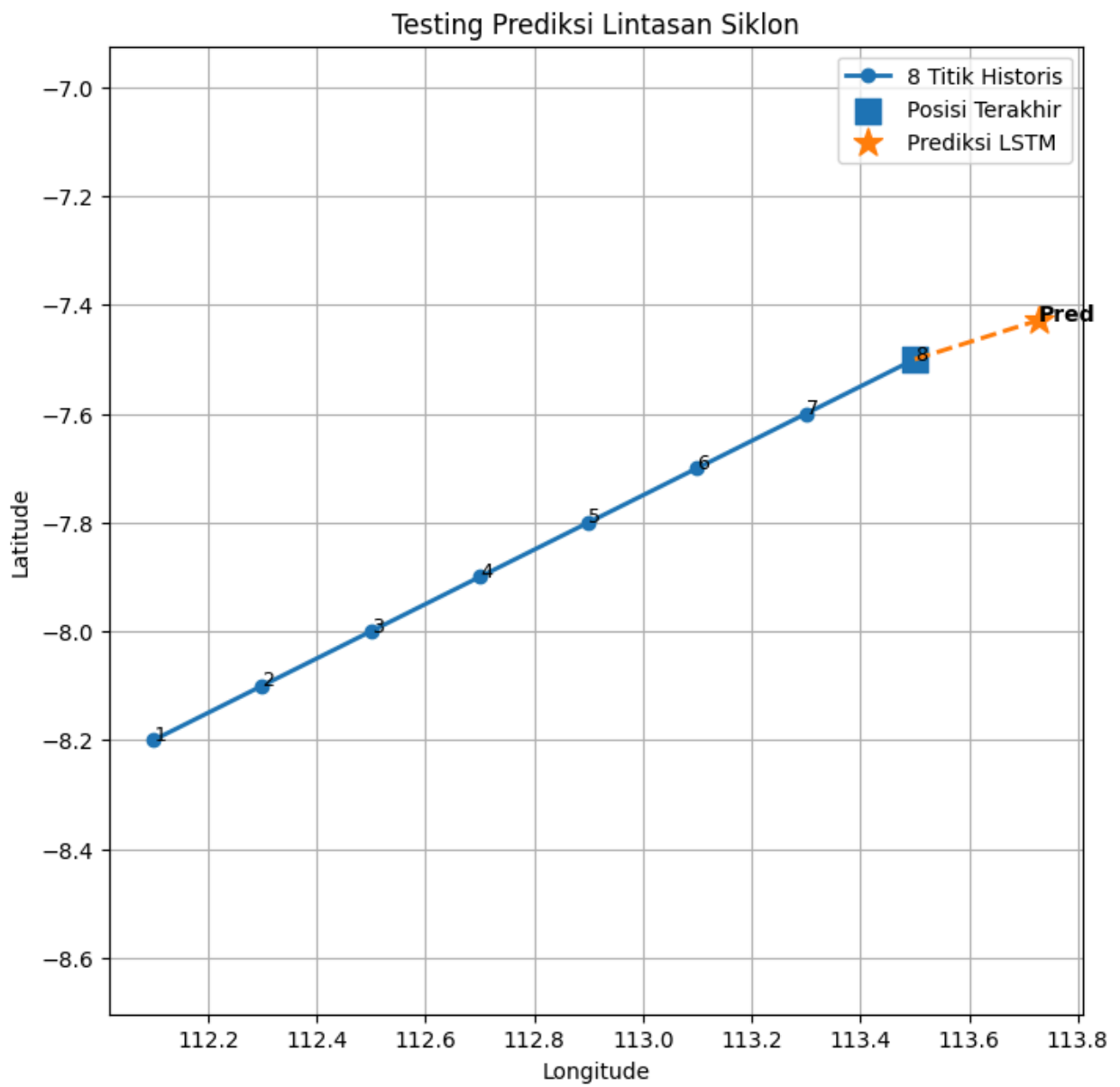
print("=" * 80)
print("Latitude Prediksi :", pred_lat)
print("Longitude Prediksi:", pred_lon)
print("=" * 80)

```

```

=====
=
VISUALISASI LINTASAN
=====
=

```



=====

=

Latitude Prediksi : -7.428184509277344

Longitude Prediksi: 113.7279281616211

=====

=